

Module 5 Report

Software Assurance and Secure SQL

EN.605.256 Modern Software Concepts

Johns Hopkins University Engineering for Professionals

Student	Abdullah Sarwar
Term	Fall 2026
Repository	git@github.com:abdulstack-ui/jhu_software_concepts.git
Final assurance commit	72d00bd
Final GitHub Actions run	37157076115
Report date	October 3, 2026

This report documents the Module 5 assurance work for the GradCafe Flask/PostgreSQL application. The final repository state uses secure SQL composition and parameterization, explicit query limits, least-privilege database access, reproducible packaging, dependency analysis, Snyk dependency scanning, Pylint enforcement, and GitHub Actions continuous integration.

Final verification: the GitHub Actions assurance job completed successfully after the last LIMIT changes. The workflow passed Pylint at 10.00/10, dependency-graph generation, Snyk dependency scanning, pytest, artifact upload, and PostgreSQL-backed integration testing. The generated coverage summary reports 100.00% coverage across 1,136 statements.

1. Reproducible Environment, Installation, and Execution

The project supports both standard pip workflows and uv-based environment synchronization. The requirements file contains the runtime dependencies needed by the application as well as development and assurance tooling such as pytest, pytest-cov, Pylint, and pydeps. The project is also installable through setup.py, which makes imports consistent between local execution, tests, and CI.

pip installation

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
python -m pip install -e .
```

uv installation

```
uv venv .venv
.\.venv\Scripts\Activate.ps1
uv pip sync requirements.txt
uv pip install -e .
```

Run the Flask application

```
$env:PYTHONPATH = (Resolve-Path .\src).Path
python -m flask --app app:create_app run
```

After the PostgreSQL environment variables are configured, the analysis page is available at <http://127.0.0.1:5000/analysis>.

Why packaging matters

The setup.py file turns the source tree into an installable Python project. An editable installation (`pip install -e .`) keeps the working source linked to the environment while giving Python a consistent import path. This reduces path-dependent behavior and the common problem where code works locally only because of a particular shell or current directory. It also makes local runs, pytest, CI, and dependency tooling use the same package definition.

2. Dependency Graph Summary

The dependency graph is rooted at `src/app.py`, which is the Flask web entry point. The application layer depends directly on the database/session layer in `models.py` and on the analytical query layer in `orm_queries.py`. The model layer uses SQLAlchemy to construct the PostgreSQL engine, map the existing applicants table, and create sessions. The ORM analysis layer uses the same Applicant model and SQLAlchemy expressions so the web page can calculate its database-backed results without handwritten SQL. The capture, scrape, clean, load, and pull modules form the data-ingestion pipeline; some of those relationships are operational rather than direct import edges because the Flask app launches the pull worker as a subprocess. Overall, the graph separates the web interface, persistence/query logic, and ingestion pipeline, which makes the security and testing boundaries easier to inspect.

The generated graph is saved as `module_5/dependency.svg` and is regenerated in CI with pydeps and Graphviz.

3. SQL Injection Defenses and Query Refactoring

The Module 5 SQL work removes unsafe construction patterns for user-influenced SQL. Dynamic identifiers are composed with psycopg's SQL composition objects, especially `sql.SQL` and `sql.Identifier`, while user-controlled values are passed separately through bound parameters. This keeps data values out of the SQL syntax itself, so malicious-looking input remains ordinary data instead of becoming executable SQL.

Statement construction is also separated from execution. A query object is built first, a separate parameter tuple is prepared, and then the database cursor executes the statement with those parameters. Security tests exercise payloads such as tautology-style and destructive-looking SQL text and verify that the payload is not embedded in the statement text.

LIMIT enforcement

All database reads are bounded. The user-configurable search limit is validated and clamped to the permitted range of 1 through 100. Fixed analytical queries use inherent LIMIT clauses; the final ORM audit contains ten `.limit(1)` calls for scalar/aggregate questions and one `.limit(5)` call for the top-five university query. This ensures the rubric requirement that every query has an inherent bound is satisfied.

Requirement	Implementation
No string interpolation of user values	Values are bound through query parameters.
Safe dynamic SQL composition	Identifiers use <code>sql.Identifier</code> and SQL structure uses <code>sql.SQL</code> .
Statement/execution separation	Statements and parameter tuples are constructed before <code>execute()</code> .
Maximum LIMIT	Configurable limit is clamped to 1-100; ORM analytics use fixed limits.
Malicious input handling	Security tests verify payload text is treated as data, not SQL syntax.

4. Least-Privilege Database Configuration

Database credentials are not hard-coded in source. The application reads either `DATABASE_URL` or the standard PostgreSQL environment variables `PGHOST`, `PGPORT`, `PGDATABASE`, `PGUSER`, and `PGPASSWORD`. The checked-in `.env.example` contains placeholders only, while the real `.env` file is ignored by Git.

The application role `module5_app` was created as a login role with no superuser, database-creation, role-creation, or inheritance capabilities. It receives `CONNECT` on the `gradcafe` database, `USAGE` on the `public` schema, and only `SELECT`, `INSERT`, and `UPDATE` on `public.applicants`. Schema `CREATE` was explicitly revoked. Privilege verification confirmed `SELECT/INSERT/UPDATE` were true while `DELETE`, `TRUNCATE`, and schema `CREATE` were false.

Representative PostgreSQL configuration

```
CREATE ROLE module5_app
WITH LOGIN
NOSUPERUSER
NOCREATEDB
NOCREATEROLE
NOINHERIT;

GRANT CONNECT ON DATABASE gradcafe TO module5_app;
GRANT USAGE ON SCHEMA public TO module5_app;
GRANT SELECT, INSERT, UPDATE
ON TABLE public.applicants
TO module5_app;

REVOKE CREATE ON SCHEMA public FROM module5_app;
```

5. Dependency Security Scan

Snyk was run against the Python dependency set in `requirements.txt`. The required open-source dependency scan tested 64 dependencies, found no vulnerable dependency paths, and reported 0 total issues. A screenshot of the result is included separately as `module_5/snyk-analysis.png`.

6. Continuous Integration and Assurance Evidence

The GitHub Actions workflow at `.github/workflows/ci.yml` runs on pushes and pull requests. The assurance job provisions PostgreSQL, installs Graphviz and Python dependencies, enforces Pylint with `--fail-under=10`, regenerates and validates `dependency.svg`, runs the Snyk dependency scan, executes pytest, and uploads the assurance artifacts.

Final CI control	Result
Pylint	10.00/10 - passed
Dependency graph	Generated and validated - passed
Snyk dependency scan	Passed
pytest + PostgreSQL integration tests	Passed
Coverage	100.00% (1,136 statements, 0 missed)
Artifact upload	Passed
GitHub Actions run	37157076115 - success

The successful-run screenshot is stored as `module_5/actions_success.png`. The CI-generated coverage evidence is stored as `module_5/coverage_summary.txt`.

7. Final Assurance Summary

Module 5 moves the project from functional database access toward explicit software assurance controls. The SQL boundary now uses safe composition and parameter binding, every read query has an enforced limit, and the application database role follows least privilege. Reproducible pip and uv setup instructions, editable packaging, dependency visualization, Snyk scanning, 10.00/10 Pylint enforcement, PostgreSQL-backed pytest execution, and 100% coverage are all integrated into the final workflow. The Canvas ZIP should be produced from the final Git commit so the submitted `module_5` directory matches the repository version exactly.